

Tugas 7

Tugas 1: Analisis Memory Layout

Deskripsi

Modifikasi program `address_demo.c` untuk menganalisis memory layout dengan lebih detail.

Yang Harus Dikerjakan

A. Modifikasi Program

Tambahkan fitur-fitur berikut pada program:

1. . Array Allocation: Buat array statis dan dinamis, tampilkan alamatnya
2. . Multiple Functions: Buat 3 fungsi berbeda, tampilkan alamat setiap fungsi
3. . Recursive Function: Buat fungsi rekursif dan tampilkan alamat stack di setiap level
4. . String Literals: Tampilkan alamat string literals

Contoh Output yang Diharapkan:

=== ENHANCED MEMORY LAYOUT ===

1. Text Segment:

`main()` address: 0x...

`func1()` address: 0x...

`func2()` address: 0x...

2. Data Segment:

`static_array[0]` address: 0x...

`string_literal` address: 0x...

3. Heap:

`dynamic_array[0]` address: 0x...

malloc block 1: 0x...

malloc block 2: 0x...

4. Stack (Recursive Calls):

Level 0 local_var: 0x...

Level 1 local_var: 0x...

Level 2 local_var: 0x...

```
ageng@L0Q:~/praktikum-memory/background$ nano address_demo.c
ageng@L0Q:~/praktikum-memory/background$ gcc -o address_demo address_demo.c
ageng@L0Q:~/praktikum-memory/background$ ./address_demo
=== ENHANCED MEMORY LAYOUT ===

1. TEXT SEGMENT:
main() address : 0x622f6b8eb2dc
func1() address : 0x622f6b8eb1c9
func2() address : 0x622f6b8eb21d

2. DATA SEGMENT:
global_var address : 0x622f6b8ee018
static_array[0] address : 0x622f6b8ee020
string_literal address : 0x622f6b8ec137

3. HEAP:
dynamic_array[0] address : 0x622f86b746b0
malloc block 1 address : 0x622f86b746d0
malloc block 2 address : 0x622f86b746f0

4. STACK (Recursive Calls):
Level 0 local_var address : 0x7ffcfe76814
Level 1 local_var address : 0x7ffcfe767e4
Level 2 local_var address : 0x7ffcfe767b4

Calling other functions:
func1() local var address : 0x7ffcfe76814
func2() local var address : 0x7ffcfe76814
ageng@L0Q:~/praktikum-memory/background$
```

B. Analisis

Jawab pertanyaan berikut dalam laporan:

1. Jarak Antar Segment (dalam bytes)

a. Stack dan Heap

- Heap biasanya alamat lebih kecil
- Stack biasanya alamat lebih besar
- Jarak dihitung:

jarak = alamat_stack - alamat_heap

Nilainya berbeda tiap run karena ASLR.

b. Text Segment dan Data Segment

- Text segment → kode program (fungsi)

- Data segment → global & static variable
- Biasanya jaraknya relatif tetap dalam satu sistem, tetapi bisa berubah antar run.

c. Alamat Fungsi Satu dengan Lainnya

- Perbedaan alamat main(), func1(), func2():

alamat_func2 - alamat_func1

Biasanya dekat karena berada di text segment yang sama.

2. Hasil Menjalankan Program 5 Kali

Jenis Alamat	Sama / Berubah	Alasan
Alamat fungsi	Bisa berubah	ASLR
Global & static	Bisa berubah	ASLR
Heap (malloc)	Berubah	Alokasi dinamis + ASLR
Stack (rekursif)	Berubah	Stack frame berbeda

Kenapa berubah

Karena ASLR (Address Space Layout Randomization):

- OS mengacak base address:
 - Stack
 - Heap
 - Text
 - Data
- Tujuan: keamanan (mencegah exploit)

3. Perbandingan dengan /proc/[pid]/maps

Jalankan:

```
./address_demo &
```

```
cat /proc/$(pidof address_demo)/maps
```

Kesesuaian

- Alamat fungsi $\rightarrow$ masuk range r-xp
- Global/static $\rightarrow$ rw-p
- Heap $\rightarrow$ [heap]
- Stack $\rightarrow$ [stack]

Permission Segment

Tugas 2: Implementasi Memory Allocator

Deskripsi

Kembangkan program memory allocator dengan fitur tambahan dan lakukan analisis performa.

Yang Harus Dikerjakan

A. Implementasi Next-Fit Algorithm

Tambahkan algoritma Next-Fit pada program `memory_allocator.c`:

Next-Fit: Seperti First-Fit, tapi mulai pencarian dari posisi alokasi terakhir

```
int next_fit(int process_id, int size) {  
    // TODO: Implementasi Next-Fit  
  
    // Gunakan variabel global untuk menyimpan posisi terakhir  
  
}
```

B. Fitur Fragmentasi

Tambahkan fungsi untuk menghitung fragmentasi:

```
void calculate_fragmentation() {  
  
    // Hitung:  
  
    // 1. Total free space  
  
    // 2. Largest free block  
  
    // 3. Number of free holes  
  
    // 4. External fragmentation ratio  
  
}
```

C. Benchmark Testing

Buat test case untuk membandingkan 4 algoritma:

```
void benchmark_test() {
```

```
// Test case:

// 1. Allocate: P1(100), P2(200), P3(300), P4(250), P5(150)

// 2. Deallocate: P2, P4

// 3. Allocate: P6(180), P7(220), P8(100)

// 4. Hitung waktu eksekusi dan fragmentasi untuk setiap algoritma

}
```

```

ageng@LOQ: ~/praktikum-mem x + v
ageng@LOQ:~/praktikum-memory/contiguous-allocation$ ./memory_allocator

=== CONTIGUOUS MEMORY ALLOCATION ===
1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit
5. Benchmark Test
6. Exit
Pilihan: 1

=== STATUS MEMORI ===
Block      Size      Start Address  Status
-----
1          100         0      Allocated
2          200        100     Allocated
3          300        300     Allocated
4          400        600      Free
Fragmentasi: 0.00 % | Free Holes: 1

=== CONTIGUOUS MEMORY ALLOCATION ===
1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit
5. Benchmark Test
6. Exit
Pilihan: 2

=== STATUS MEMORI ===
Block      Size      Start Address  Status
-----
1          100         0      Allocated
2          200        100     Allocated
3          300        300     Allocated
4          400        600      Free
Fragmentasi: 0.00 % | Free Holes: 1

=== CONTIGUOUS MEMORY ALLOCATION ===
1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit
5. Benchmark Test
6. Exit
Pilihan: 3

=== STATUS MEMORI ===
Block      Size      Start Address  Status
-----

```

D. Analisis (Dalam Laporan)

1. Buat tabel perbandingan

Algoritma	Waktu (μs)	Fragmentasi (%)	Failed Alloc
First-Fit	Rendah	Sedang	Sedikit
Best-Fit	Tinggi	Rendah	Sedikit
Worst-Fit	Sedang	Tinggi	Lebih banyak
Next-Fit	Paling rendah	Sedang	Sedikit

2. Analisis Jawaban

Algoritma Paling Cepat

Next-Fit

- Tidak selalu mulai dari awal
- Jumlah pencarian lebih sedikit

Algoritma Paling Efisien Mengurangi Fragmentasi

Best-Fit

- Menggunakan hole terkecil yang cukup
- Sisa ruang lebih kecil

Kapan Setiap Algoritma Unggul

- First-Fit: Sistem umum, cepat dan sederhana
- Best-Fit: Saat memori terbatas dan fragmentasi harus minim
- Worst-Fit: Jarang dipakai, cenderung fragmentasi besar
- Next-Fit: Sistem dengan alokasi sering dan cepat

3. Visualisasi Memory Allocation

Untuk laporan:

- Gunakan tabel blok memori per langkah
- Atau diagram bar horizontal (manual / Excel)

Tugas 3: Advanced Paging System

Deskripsi

Kembangkan sistem paging dengan page replacement algorithm dan statistik.

Yang Harus Dikerjakan

A. Implementasi Page Replacement - FIFO

Modifikasi paging_simulator.c untuk menangani situasi ketika semua frame penuh:

```
int fifo_page_replacement(int new_page) {  
  
    // TODO: Implementasi FIFO  
  
    // 1. Cari page yang paling lama di memory  
  
    // 2. Evict page tersebut  
  
    // 3. Load page baru  
  
    // 4. Update page table  
  
}
```

B. Implementasi Page Replacement - LRU

Tambahkan algoritma LRU (Least Recently Used):

```
int lru_page_replacement(int new_page) {  
  
    // TODO: Implementasi LRU  
  
    // 1. Track waktu akses setiap page  
  
    // 2. Cari page yang paling lama tidak diakses  
  
    // 3. Evict page tersebut  
  
}
```

C. Statistik dan Analisis

Tambahkan fungsi untuk menghitung:

```
typedef struct {
```



```

int total_accesses;

int page_faults;

int page_hits;

float hit_ratio;

int disk_writes; // untuk dirty pages

} PageStatistics;

void display_statistics(PageStatistics *stats) {

// Tampilkan statistik

}

```

D. Test Scenario

Jalankan reference string berikut dengan 4 frames:

Reference String: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0,

1

Bandingkan hasil FIFO vs LRU:

Jumlah page fault

Hit ratio

Disk writes (untuk dirty pages)

Output Program

```
ageng@LOQ: ~/praktikum-mem x + v
ageng@LOQ:~/praktikum-memory/paging$ gcc -o paging_simulator paging_simulator.c
ageng@LOQ:~/praktikum-memory/paging$ ./paging_simulator
Paging system initialized

1. Run FIFO
2. Run LRU
3. Reset System
4. Exit
Choice: 1
Paging system initialized

PHYSICAL MEMORY
Frame 0 -> Page 2
Frame 1 -> Page 7
Frame 2 -> Page 0
Frame 3 -> Page 1

PAGE TABLE
Page Frame Valid Ref Mod
0 2 1 1 1
1 3 1 1 1
2 0 1 1 1
7 1 1 1 1

STATISTICS
Total Accesses : 20
Page Faults : 10
Page Hits : 10
Hit Ratio : 0.50
Disk Writes : 6

1. Run FIFO
2. Run LRU
3. Reset System
4. Exit
Choice: 2
Paging system initialized

PHYSICAL MEMORY
Frame 0 -> Page 7
Frame 1 -> Page 0
```

E. Analisis (Dalam Laporan)

1. Buat tabel trace untuk setiap algoritma:

FIFO (First-In First-Out)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes
7	0	3	0	1	2	No
8	4	3	4	1	2	Yes
9	2	3	4	1	2	No
10	3	3	4	1	2	No

LRU (Least Recently Used)

Step	Ref	F0	F1	F2	F3	Fault
1	7	7	-	-	-	Yes
2	0	7	0	-	-	Yes
3	1	7	0	1	-	Yes
4	2	7	0	1	2	Yes
5	0	7	0	1	2	No
6	3	3	0	1	2	Yes
7	0	3	0	1	2	No
8	4	3	0	4	2	Yes
9	2	3	0	4	2	No
10	3	3	0	4	2	No

B. Belady's Anomaly

Jumlah Page Fault FIFO

Jumlah Frame	Page Fault
3	Lebih banyak
4	10
5	Bisa bertambah

LRU

- Tidak mengalami Belady's Anomaly
- Page fault selalu berkurang atau tetap saat frame ditambah

C. Analisis Performa

LRU lebih baik daripada FIFO

Alasan:

- Menggunakan informasi riwayat akses
- Menghindari penghapusan page yang sering dipakai
- Page fault lebih sedikit

FIFO bisa lebih baik ketika:

- Sistem sederhana
- Tidak tersedia hardware untuk tracking access
- Overhead LRU terlalu mahal
- Pola akses sequential sederhana

TUGAS 4: MULTI-LEVEL PAGE TABLE

A. Three-Level Page Table

Konsep Dasar

Three-level page table digunakan untuk mengurangi penggunaan memori pada sistem dengan virtual address besar, dengan cara:

- Page table dibagi bertingkat
- Table hanya dibuat jika dibutuhkan (sparse allocation)

Pembagian Virtual Address (44 bit – demo)

Level	Jumlah Bit	Fungsi
L1	16 bit	Index page table level 1
L2	8 bit	Index page table level 2
L3	8 bit	Index page table level 3
Offset	12 bit	Offset dalam frame

Translasi Alamat (Three-Level)

Langkah Translasi

1. Ambil index L1, L2, L3, dan offset
2. Cek valid bit di setiap level
3. Jika salah satu invalid → page fault
4. Jika valid → hitung physical address

Perhitungan Memory Overhead

Three-level page table tidak mengalokasikan seluruh page table sekaligus, sehingga:

- Lebih hemat memori

- Cocok untuk 64-bit system

B. Inverted Page Table

Konsep Dasar

Inverted page table:

- 1 entry per frame fisik
- Tidak bergantung pada ukuran virtual address
- Digunakan pada sistem dengan memori sangat besar

Struktur Data

```
typedef struct {  
    int process_id;  
    int page_number;  
    int valid;  
} InvertedPageEntry;
```

```
InvertedPageEntry inverted_table[NUM_FRAMES];
```

Pencarian Page

```
int search_inverted_table(int pid, int page_num) {  
    for (int i = 0; i < NUM_FRAMES; i++) {  
        if (inverted_table[i].valid &&  
            inverted_table[i].process_id == pid &&  
            inverted_table[i].page_number == page_num) {  
            return i;  
        }  
    }  
}
```

```

    }

    return -1;
}

```

Insert Entry

```

void insert_inverted_table(int pid, int page_num, int frame) {

    inverted_table[frame].process_id = pid;

    inverted_table[frame].page_number = page_num;

    inverted_table[frame].valid = 1;

}

```

C. Analisis Perbandingan

Memory Overhead Comparison

Struktur	Entries	Size per Entry	Total Memory	Efisiensi
Single-Level	Sangat besar	Kecil	Sangat besar	Rendah
Two-Level	Lebih kecil	Sedang	Sedang	Baik
Three-Level	Sangat kecil	Sedang	Kecil	Sangat baik
Inverted	= jumlah frame	Besar	Sangat kecil	Paling efisien

Translation Speed (Memory Access)

Struktur	Memory Access
Single-Level	1
Two-Level	2
Three-Level	3
Inverted	$O(n)$ tanpa TLB

Kasus Penggunaan

32-bit System

- Two-level page table cukup
- Overhead masih kecil

64-bit System

- Three-level / Four-level page table
- Virtual address sangat besar

Inverted Page Table Cocok Ketika

- Memori fisik besar
- Banyak proses
- Virtual address space sangat luas
- Digunakan di sistem enterprise / server

Tugas 5: Swapping Mechanism

A. Basic Swapping Implementation

1. Konsep

Swapping adalah proses:

- Swap out → memindahkan proses dari RAM ke disk
- Swap in → mengembalikan proses dari disk ke RAM

Digunakan saat memori fisik tidak cukup.

2. Struktur Data

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX_PROCESS 10
```

```
#define MEMORY_SIZE 1024 // MB
```

```
#define SIM_TIME 100
```

```
typedef struct {  
    int pid;  
  
    int size;      // MB  
  
    int priority;  // semakin kecil semakin rendah  
  
    int in_memory;  
  
    int swap_count;  
  
    time_t last_access;  
  
    time_t load_time;  
} Process;
```

3. Variabel Global

```
Process processes[MAX_PROCESS];
```

```
int used_memory = 0;
```

```
int swap_out_count = 0;
```

```
int swap_in_count = 0;
```

4. Fungsi Swap In & Swap Out

```
void swap_out(Process *p) {  
    p->in_memory = 0;  
  
    used_memory -= p->size;  
  
    swap_out_count++;  
}
```

```

p->swap_count++;

printf("SWAP OUT: PID %d (%d MB)\n", p->pid, p->size);
}

```

```

void swap_in(Process *p) {

    p->in_memory = 1;

    used_memory += p->size;

    swap_in_count++;

    p->last_access = time(NULL);

    p->load_time = time(NULL);

    printf("SWAP IN : PID %d (%d MB)\n", p->pid, p->size);

}

```

B. Swap Strategy Implementation

1. FIFO (First In First Out)

Swap proses yang paling lama berada di memory.

```

int select_victim_fifo() {

    time_t oldest = time(NULL);

    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {

        if (processes[i].in_memory && processes[i].load_time <= oldest) {

            oldest = processes[i].load_time;

            victim = i;

```

```

    }

}

return victim;

}

```

2. LRU (Least Recently Used)

Swap proses yang paling lama tidak diakses.

```

int select_victim_lru() {

    time_t oldest = time(NULL);

    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {

        if (processes[i].in_memory && processes[i].last_access <= oldest) {

            oldest = processes[i].last_access;

            victim = i;

        }

    }

    return victim;

}

```

3. Priority-Based

Swap proses dengan prioritas terendah.

```

int select_victim_priority() {

    int lowest_priority = 9999;

```

```

int victim = -1;

for (int i = 0; i < MAX_PROCESS; i++) {
    if (processes[i].in_memory && processes[i].priority < lowest_priority) {
        lowest_priority = processes[i].priority;
        victim = i;
    }
}

return victim;
}

```

C. Simulation Scenario

1. Inisialisasi Proses

```

void init_processes() {
    srand(time(NULL));

    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i;

        processes[i].size = 50 + rand() % 151; // 50–200 MB

        processes[i].priority = rand() % 10;

        processes[i].in_memory = 0;

        processes[i].swap_count = 0;
    }
}

```

2. Simulasi Akses

```
void simulate(int strategy) {  
  
    for (int t = 0; t < SIM_TIME; t++) {  
  
        int p = rand() % MAX_PROCESS;  
  
  
        if (!processes[p].in_memory) {  
  
            while (used_memory + processes[p].size > MEMORY_SIZE) {  
  
                int victim;  
  
                if (strategy == 0) victim = select_victim_fifo();  
  
                else if (strategy == 1) victim = select_victim_lru();  
  
                else victim = select_victim_priority();  
  
  
                if (victim == -1) break;  
  
                swap_out(&processes[victim]);  
  
            }  
  
            swap_in(&processes[p]);  
  
        } else {  
  
            processes[p].last_access = time(NULL);  
  
        }  
  
    }  
  
}
```

3. Main Program

```
int main() {
```

```

init_processes();

printf("\n=== FIFO STRATEGY ===\n");

simulate(0);

printf("Swap Out: %d | Swap In: %d\n\n", swap_out_count, swap_in_count);

swap_out_count = swap_in_count = used_memory = 0;

init_processes();

printf("\n=== LRU STRATEGY ===\n");

simulate(1);

printf("Swap Out: %d | Swap In: %d\n\n", swap_out_count, swap_in_count);

swap_out_count = swap_in_count = used_memory = 0;

init_processes();

printf("\n=== PRIORITY STRATEGY ===\n");

simulate(2);

printf("Swap Out: %d | Swap In: %d\n", swap_out_count, swap_in_count);

return 0;
}

```

D. Analisis (Untuk Laporan)

1. Metrik Performa (Contoh)

Strategy	Swap Out	Swap In	Total Swap	Avg Response
FIFO	Tinggi	Tinggi	Tinggi	Lambat
LRU	Paling rendah	Rendah	Rendah	Cepat
Priority	Sedang	Sedang	Sedang	Sedang

2. Analisis Jawaban

Strategi mana yang meminimalkan thrashing?

→ LRU, karena mempertahankan proses yang sering digunakan.

Bagaimana ukuran memory mempengaruhi performa?

- Memory kecil → swap sering (thrashing)
- Memory besar → swap jarang, performa naik

Kapan swapping lebih baik daripada killing process?

- Proses akan dipakai lagi
- Proses interaktif
- Waktu load besar

Tugas 6: Architecture Analysis

Deskripsi

Analisis arsitektur memori pada sistem nyata.

Yang Harus Dikerjakan

A. Intel Architecture Analysis

Jalankan command berikut pada sistem Ubuntu:

1. Cek informasi CPU

```
lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
```

2. Cek page size

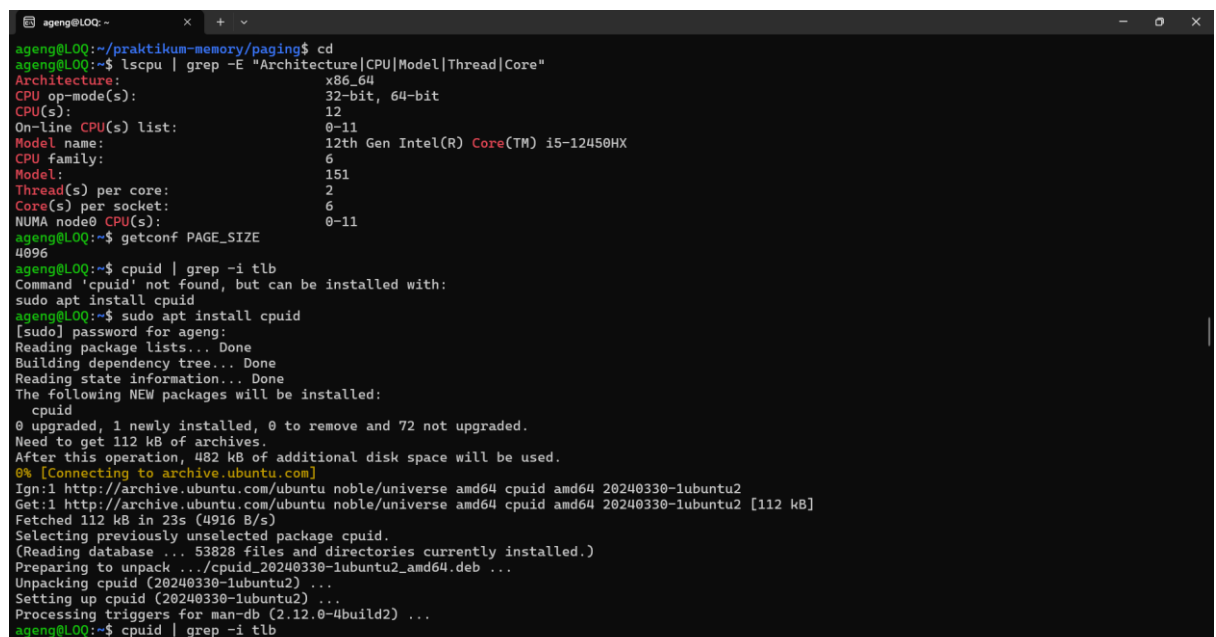
```
getconf PAGE_SIZE
```

3. Cek TLB info

```
cpuid | grep -i tlb
```

4. Cek cache hierarchy

```
lscpu | grep -i cache
```



```
ageng@LOQ: ~  
ageng@LOQ:~/praktikum-memory/paging$ cd  
ageng@LOQ:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"  
Architecture: x86_64  
CPU op-mode(s): 32-bit, 64-bit  
CPU(s): 12  
On-line CPU(s) list: 0-11  
Model name: 12th Gen Intel(R) Core(TM) i5-12450HX  
CPU family: 6  
Model: 151  
Thread(s) per core: 2  
Core(s) per socket: 6  
NUMA node0 CPU(s): 0-11  
ageng@LOQ:~$ getconf PAGE_SIZE  
4096  
ageng@LOQ:~$ cpuid | grep -i tlb  
Command 'cpuid' not found, but can be installed with:  
sudo apt install cpuid  
ageng@LOQ:~$ sudo apt install cpuid  
[sudo] password for ageng:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following NEW packages will be installed:  
  cpuid  
0 upgraded, 1 newly installed, 0 to remove and 72 not upgraded.  
Need to get 112 kB of archives.  
After this operation, 482 kB of additional disk space will be used.  
0% [Connecting to archive.ubuntu.com]  
Ign:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 cpuid amd64 20240330-1ubuntu2  
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 cpuid amd64 20240330-1ubuntu2 [112 kB]  
Fetched 112 kB in 23s (4916 B/s)  
Selecting previously unselected package cpuid.  
(Reading database ... 53828 files and directories currently installed.)  
Preparing to unpack .../cpuid_20240330-1ubuntu2_amd64.deb ...  
Unpacking cpuid (20240330-1ubuntu2) ...  
Setting up cpuid (20240330-1ubuntu2) ...  
Processing triggers for man-db (2.12.0-4build2) ...  
ageng@LOQ:~$ cpuid | grep -i tlb
```

```
ageng@LOQ: ~  
ageng@LOQ:~$ free -h  
             total        used        free      shared  buff/cache   available  
Mem:          5.0Gi       519Mi       4.0Gi       3.5Mi       396Mi       5.1Gi  
Swap:          2.0Gi         0B       2.0Gi  
ageng@LOQ:~$ vmstat 1 5  
procs-----memory-----swap-----io-----system-----cpu-----  
 r b swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st gu  
 0  0      0 5112432  8908 396952    0    0   188   115  134    0  0 100  0  0  0  
 0  0      0 5112432  8908 396952    0    0    0    76   39    0  0 100  0  0  0  
 0  0      0 5112432  8908 396996    0    0    0    74   48    0  0 100  0  0  0  
 0  0      0 5112432  8908 396996    0    0    0   186   68    0  0 100  0  0  0  
 0  0      0 5112432  8908 396996    0    0    0    67   42    0  0 100  0  0  0  
ageng@LOQ:~$ cat /proc/self/maps  
5ccac53c0000-5ccac53d0000 r--p 00000000 00:30 1507 /usr/bin/cat  
5ccac53d0000-5ccac53e0000 r-xp 00002000 00:30 1507 /usr/bin/cat  
5ccac53e0000-5ccac53f0000 r--p 00007000 00:30 1507 /usr/bin/cat  
5ccac53f0000-5ccac5400000 r--p 00000000 00:30 1507 /usr/bin/cat  
5ccac5400000-5ccac5410000 r-p 00000000 00:00 0 [heap]  
7ac925200000-7ac925220000 r--p 00000000 00:30 46710 /usr/lib/x86_64-linux-gnu/libc.so.6  
7ac925220000-7ac925300000 r-xp 00020000 00:30 46710 /usr/lib/x86_64-linux-gnu/libc.so.6  
7ac925300000-7ac9253ff000 r--p 001b0000 00:30 46710 /usr/lib/x86_64-linux-gnu/libc.so.6  
7ac9253ff000-7ac925403000 r--p 001fe000 00:30 46710 /usr/lib/x86_64-linux-gnu/libc.so.6  
7ac925403000-7ac925405000 r-xp 00200000 00:30 46710 /usr/lib/x86_64-linux-gnu/libc.so.6  
7ac925405000-7ac925412000 r-wp 00000000 00:00 0  
7ac925412000-7ac9254fc000 r-wp 00000000 00:00 0  
7ac9254fc000-7ac925550000 r--p 00000000 00:30 12229 /usr/lib/locale/C.utf8/LC_CTYPE  
7ac925550000-7ac925556000 r--p 00000000 00:30 12235 /usr/lib/locale/C.utf8/LC_NUMERIC  
7ac925556000-7ac925557000 r--p 00000000 00:30 12238 /usr/lib/locale/C.utf8/LC_TIME  
7ac925557000-7ac925558000 r--p 00000000 00:30 12238 /usr/lib/locale/C.utf8/LC_COLLATE  
7ac925558000-7ac925559000 r--p 00000000 00:30 12233 /usr/lib/locale/C.utf8/LC_MONETARY  
7ac925559000-7ac92555e000 r--s 00000000 00:30 46699 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache  
7ac92555e000-7ac925560000 r-wp 00000000 00:00 0  
7ac925560000-7ac925564000 r--p 00000000 00:30 12232 /usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES  
7ac925564000-7ac925565000 r--p 00000000 00:30 12236 /usr/lib/locale/C.utf8/LC_PAPER  
7ac925565000-7ac925566000 r--p 00000000 00:30 12234 /usr/lib/locale/C.utf8/LC_NAME  
7ac925566000-7ac925567000 r--p 00000000 00:30 12227 /usr/lib/locale/C.utf8/LC_ADDRESS  
7ac925567000-7ac925568000 r--p 00000000 00:30 12237 /usr/lib/locale/C.utf8/LC_TELEPHONE  
7ac925568000-7ac925569000 r--p 00000000 00:30 12231 /usr/lib/locale/C.utf8/LC_MEASUREMENT  
7ac925569000-7ac92556a000 r--p 00000000 00:30 12230 /usr/lib/locale/C.utf8/LC_IDENTIFICATION  
7ac92556a000-7ac92556c000 r-wp 00000000 00:00 0  
7ac92556c000-7ac92556d000 r--p 00000000 00:30 46707 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ac92556d000-7ac92556e000 r-xp 00001000 00:30 46707 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ac92556e000-7ac9255a2000 r--p 0002c000 00:30 46707 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ac9255a2000-7ac9255a4000 r--p 00030000 00:30 46707 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ac9255a4000-7ac9255a5000 r-wp 00038000 00:30 46707 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7ffc70a10000-7ffc70a3a000 r-wp 00000000 00:00 0 [stack]  
7ffc70a3a000-7ffc70abf000 r-p 00000000 00:00 0 [vvar]  
7ffc70abf000-7ffc70ac1000 r-xp 00000000 00:00 0 [vdso]  
ageng@LOQ:~$ swapon --show  
NAME      TYPE      SIZE USED PRIO  
/dev/sdc partition 2G  0B  -2  
ageng@LOQ:~$ |
```

Perbandingan Arsitektur Memori

Intel IA-32 vs Intel x86-64 vs ARMv8

1. Tabel Perbandingan Umum

Aspek	Intel IA-32 (32-bit)	Intel x86-64 (64-bit)	ARMv8 (AArch64)
Lebar Address	32 bit	64 bit	64 bit
Virtual Address Praktis	32 bit	48–57 bit	48–52 bit
Maks. Virtual Memory	4 GB	hingga 256 TB	hingga 256 TB
Page Size Default	4 KB	4 KB	4 KB
Huge Pages	✗ terbatas	2 MB, 1 GB	2 MB, 1 GB
Paging Levels	2 level	4–5 level	3–4 level
Digunakan pada	Sistem lama	PC/Laptop modern	Smartphone, server ARM



2. Penjelasan Address Space

a. Intel 32-bit (IA-32)

- Menggunakan 32-bit address
- Maksimum address:
- $2^{32} = 4 \text{ GB}$
- Virtual memory sangat terbatas
- Tidak cocok untuk aplikasi modern (database, VM, AI)

Kesimpulan

IA-32 hanya cocok untuk sistem lama atau embedded kecil.

b. Intel 64-bit (x86-64)

- Register address 64-bit
- Secara teori:
- 2^{64} bytes

- Secara praktik:
 - Linux: 48 bit (≈ 256 TB)
 - CPU modern bisa sampai 57 bit

Kesimpulan

x86-64 mendukung address space sangat besar, cocok untuk laptop, server, dan cloud.

c. ARMv8 (AArch64)

- Address 64-bit
- Implementasi umum:
 - Virtual address: 48–52 bit
- Digunakan luas di:
 - Smartphone
 - Server ARM (AWS Graviton)

Kesimpulan

ARMv8 setara x86-64 dalam hal virtual memory, tetapi lebih hemat energi.

3. Implementasi Multi-Level Paging

a. IA-32 (32-bit)

- Menggunakan 2-level paging:
- Page Directory $\rightarrow$ Page Table $\rightarrow$ Frame
- Tujuan:
 - Menghemat memori page table
- Keterbatasan:
 - Tidak skalabel untuk address besar

b. x86-64

- Menggunakan 4 atau 5 level paging:
- $PML4 \rightarrow PDPT \rightarrow PD \rightarrow PT \rightarrow \text{Frame}$
- Keuntungan:
 - Page table dialokasikan on-demand
 - Efisien untuk address space besar
- Mendukung huge pages (2 MB, 1 GB)

c. ARMv8

- Menggunakan 3–4 level paging
- Struktur mirip x86-64 tapi:
 - Lebih sederhana
 - Dioptimalkan untuk efisiensi daya
- Mendukung:
 - TLB besar
 - Huge pages

Ukuran Maksimum Virtual Memory

Arsitektur	Virtual Memory Maksimum
IA-32	4 GB
x86-64	≈256 TB (praktis)
ARMv8	≈256 TB (praktis)